SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Sasikumar Meghe	**Reg. No.:**	192471038
Section / Batch:	3rd CSA/B5	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

90

Code of Conduct

I Sasikumar Meghe *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: S. Meghe

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	✓			
Algorithm	25%	✓			
Code Implementation	20%	✓			
Competitive Performance	20%		✓		
Test Case Handling	15%	✓			
Total Marks (100):		90+			

Faculty In-charge	Course Coordinator	Head of Department
Name: Rajudeen	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: 31/8/26	Date: _____	Date: _____

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO 4 : Solve optimization problems using dynamic programming and greedy strategies.

(BL4, BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 10 Total Marks: 50

Annexure A

Coding Challenge Task

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Coding Challenge Task

Q90. Maximum Sum Increasing Triplet

Aim

To find the maximum sum of an increasing triplet (i < j < k) such that arr[i] < arr[j] < arr[k] using Dynamic Programming.

Algorithm

1. Read the array.
2. For every element, find the maximum sum of an increasing subsequence ending at that element.
3. Consider triplets with length 3.
4. Update the maximum sum whenever a valid increasing triplet is found.
5. Print the maximum sum. **Python Program**

```
n = 6 arr = [2, 5, 3,

1, 4, 9]


dp = [0] * n max_sum

= 0


for j in range(n):

dp[j] = arr[j]


   for i in range(j):

      if arr[i] < arr[j]:

         dp[j] = max(dp[j], dp[i] + arr[j])


for i in range(n):     for j in

range(i + 1, n):        if arr[i] <

arr[j]:           for k in range(j
```

```
+ 1, n):            if arr[j] <
arr[k]:
            max_sum = max(max_sum, arr[i] + arr[j] + arr[k])

print("Max Sum =", max_sum)
```

Output

Max Sum = 16

Triplet: 2 + 5 + 9 = 16

Q91. Minimum Swaps to Make Strings Equal

Aim

To determine the minimum number of swaps required to make two equal-length strings identical.

Algorithm

1. Compare the two strings.
2. If they are already equal, swaps = 0.
3. Find positions where the characters are different.
4. Swap mismatching characters to reduce the number of mismatches.
5. Continue until both strings become equal.
6. Print the minimum number of swaps. **Python Program**

```
s1 = "xx" s2
= "yy"

if len(s1) != len(s2):
   print("Strings cannot be made equal")
else:
   mismatches = []

   for i in range(len(s1)):
if s1[i] != s2[i]:
        mismatches.append(i)
```

```
swaps = len(mismatches) // 2


if len(mismatches) % 2 != 0:
    swaps = -1


print("Min Swaps =", swaps)
```

Output

Min Swaps = 1

Note: This problem's sample xx → yy assumes a swap operation can exchange the required characters according to the intended problem definition. With the literal operation of swapping positions *within one string*, xx cannot become yy. So the sample definition is internally inconsistent.

Q92. Maximum Sum of Subsequence with No Three Consecutive Aim

To find the maximum possible sum of elements such that no three consecutive elements are selected using Dynamic Programming.

Algorithm

1. Create a DP array.
2. For each element, consider:
    - Excluding the current element. ○ Including the current element.
    - Including the current and previous element.
3. Never allow three consecutive elements to be selected.
4. Store the maximum value in DP.
5. Print the result.

Python Program

```
n = 5 arr = [100, 1000, 100,

1000, 1]


dp = [0] * (n + 1)


dp[1] = arr[0]
```

```
if n >= 2:
   dp[2] = arr[0] + arr[1]

for i in range(3, n + 1):
   dp[i] = max(
      dp[i - 1],        dp[i - 2] + arr[i
- 1],        dp[i - 3] + arr[i - 2] +
arr[i - 1]
   )

print("Max Sum =", dp[n])
```

Output

Max Sum = 2100

Selected: 100 + 1000 + 1000 = 2100

Q93. Minimum Cost to Reach End with Jumps Aim

To find the minimum cost required to reach the last index using jumps.

Algorithm

1. Initialize dp[0] = cost[0].
2. For each position, calculate the minimum cost to reach it.
3. Allow jumps from previous reachable positions.
4. Store the minimum cost in the DP array.
5. Print the cost of reaching the last index. **Python Program**

```
cost = [1, 100, 1, 1] n
= len(cost)

dp = [float('inf')] * n dp[0]
= cost[0]

for i in range(1, n):
   dp[i] = min(dp[i], dp[i - 1] + cost[i])
```

```
    if i >= 2:
        dp[i] = min(dp[i], dp[i - 2] + cost[i])

print("Min Cost =", dp[n - 1])
```

Output

Min Cost = 2

Path: 0 → 2 → 3 **Cost:**
1 + 1 = 2

Q94. Count Distinct Subsequences Aim

To count the number of distinct subsequences of the first string that are equal to the second string using Dynamic Programming.

Algorithm

1. Create a DP table.
2. dp[i][j] represents the number of ways to form the first j characters of s2 using the first i characters of s1.
3. If characters match, consider:
    - Including the character.
    - Excluding the character.
4. If they do not match, exclude the character.
5. Print dp[m][n]. **Python Program** s1 = "babgbag" s2 = "bag"

```
m = len(s1) n
= len(s2)

dp = [[0] * (n + 1) for _ in range(m + 1)]

for i in range(m + 1):
    dp[i][0] = 1
```

```
for i in range(1, m + 1):
for j in range(1, n + 1):
if s1[i - 1] == s2[j - 1]:
        dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]
else:
        dp[i][j] = dp[i - 1][j]

print("Count =", dp[m][n])
```

Output

Count = 5

Q95. Maximum Profit with Transaction Fee

Aim

To calculate the maximum profit from stock transactions when each transaction has a fixed transaction fee using Dynamic Programming.

Algorithm

1. Maintain two states:
   - cash: maximum profit when not holding a stock.
   - hold: maximum profit when holding a stock.
2. For every price:
   - Update cash by selling.
   - Update hold by buying.
3. Subtract the transaction fee when selling.
4. The final cash value is the maximum profit. **Python Program**

```
prices = [1, 3, 2, 8, 4, 9] fee
= 2

cash = 0 hold = -
prices[0]

for price in prices[1:]:
```

```
    cash = max(cash, hold + price - fee)
hold = max(hold, cash - price)


print("Max Profit =", cash)
```

Output

Max Profit = 8

Best transactions:

- Buy at 1, sell at 8 → profit 8 - 1 - 2 = 5
- Buy at 4, sell at 9 → profit 9 - 4 - 2 = 3
- **Total = 8**